OPERATING SYSTEMS

Complete Bible — 2026

Beginner · Intermediate · Advanced · Pro/Expert

Processes · Memory · Scheduling · Synchronization · File Systems

Kernel Internals · Virtualization · Security · Distributed Systems

WHAT'S INSIDE:

40 fully-detailed chapters across 4 mastery levels

Complete code examples in C, Python, and pseudocode

Real-world Linux & Windows kernel case studies

Generated: 2026

CONTENTS AT A GLANCE

QUICK REFERENCE

PART I

BEGINNER

For those starting from zero — no prior OS knowledge assumed

Chapter 1

What Is an Operating System?

Why Do We Need an Operating System?

Imagine a computer with no operating system. Every program you write would need to know exactly how to talk to the specific hard drive, the specific display, the specific keyboard. If you swapped the hard drive for a different brand, your program would break. The operating system (OS) solves this problem by sitting between your programs and the hardware, providing a consistent interface.

An **operating system** is software that manages computer hardware, provides services for application programs, and acts as an intermediary between users and hardware. Think of it as a government for your computer: it sets rules, manages resources, and ensures different programs can coexist without stepping on each other.

Core Responsibilities of an OS

Responsibility	What It Does	Real Example
Process Management	Creates, schedules, and terminates programs	Running Chrome and Spotify simultaneously
Memory Management	Allocates and reclaims RAM for programs	Each app gets its own memory space
File System	Organizes data on disks into files and folders	Saving a document to /home/user/docs/
Device Management	Controls hardware via drivers	Sending a document to a printer
Security	Controls who can access what	Login passwords, file permissions
User Interface	CLI or GUI for user interaction	Terminal shell, Windows desktop

Kernel Mode vs User Mode

CPUs have (at minimum) two privilege levels. **Kernel mode** (ring 0 on x86) can execute any instruction and access all memory. **User mode** (ring 3) is restricted — programs cannot directly touch hardware or other programs' memory. When a program needs OS services (like reading a file), it makes a **system call**, which switches the CPU to kernel mode, performs the operation, then returns to user mode.

```
// Simplified: what happens when you call read() in C
// 1. Your program calls read(fd, buffer, size)
// 2. C library sets up registers: syscall number, arguments
// 3. Executes 'syscall' instruction (x86-64) or 'int 0x80' (x86-32)
// 4. CPU switches to kernel mode, jumps to syscall handler
// 5. Kernel reads data from disk into buffer
// 6. CPU returns to user mode, read() returns bytes read
```

Types of Operating Systems

Type	Key Feature	Examples
Batch	Jobs queued and run without interaction	IBM OS/360, early mainframes
Time-sharing	Multiple users share CPU via time slices	Unix, Multics
Real-time (RTOS)	Guaranteed response within deadlines	QNX, VxWorks, FreeRTOS
Distributed	Multiple machines appear as one system	Google's Borg, Plan 9
Mobile	Touch-optimized, power-efficient	Android (Linux), iOS (XNU/Darwin)

Type	Key Feature	Examples
Embedded	Runs on specialized hardware	Router firmware, smart thermostats

A Brief History

1950s: No OS — programs ran directly on hardware, one at a time. 1960s: Batch processing and time-sharing emerged (Multics). 1969: Ken Thompson and Dennis Ritchie created Unix at Bell Labs — the ancestor of Linux, macOS, Android, and iOS. 1970s-80s: Personal computers brought MS-DOS and Mac OS. 1991: Linus Torvalds released the Linux kernel. 2000s-present: Mobile OSes (Android 2008, iOS 2007) and cloud computing reshaped the landscape. Today, Linux powers most of the internet's servers, Android runs on 3+ billion devices, and Windows dominates desktops.

Key Takeaway

The OS is the most important piece of software on a computer. Every application you use — from web browsers to games to databases — relies on the OS for process management, memory, file access, and hardware interaction. Understanding how the OS works is fundamental to becoming a strong software engineer.

Chapter 2

Computer Hardware Basics for OS

Why Hardware Knowledge Matters

You cannot understand operating systems without understanding the hardware they manage. The OS is the software layer that bridges application programs and physical components. This chapter covers the hardware concepts every OS student needs.

The CPU (Central Processing Unit)

The CPU executes instructions. Key concepts: **Registers** are tiny, ultra-fast storage (nanoseconds) inside the CPU. The **Program Counter (PC)** holds the address of the next instruction. The **Stack Pointer (SP)** points to the top of the current stack. **Instruction cycle**: Fetch instruction → Decode → Execute → Store result. Modern CPUs are pipelined (multiple instructions in flight) and superscalar (multiple execution units).

Component	Speed	Size	Purpose
Registers	~0.3 ns	~1 KB total	Hold current data and addresses
L1 Cache	~1 ns	32-64 KB per core	Frequently used data
L2 Cache	~4 ns	256 KB-1 MB per core	Larger fast cache
L3 Cache	~10 ns	4-64 MB shared	Shared across cores
RAM (DRAM)	~100 ns	8-128 GB typical	Main memory
SSD	~25-100 µs	256 GB-4 TB	Persistent storage
HDD	~5-10 ms	1-20 TB	Bulk persistent storage

Notice the massive speed gaps. RAM is ~100x slower than L1 cache. Disk is ~100,000x slower than RAM. The OS must manage these gaps efficiently.

Interrupts

An **interrupt** is a signal to the CPU that something needs attention. When an interrupt fires: (1) CPU stops current work, (2) saves its state, (3) jumps to an interrupt handler (a kernel function), (4) handles the event, (5) restores state and resumes. Types: **Hardware interrupts** come from devices (keyboard press, disk read complete, timer tick). **Software interrupts** (traps) come from programs (system calls, divide-by-zero errors).

DMA (Direct Memory Access)

Without DMA, the CPU would have to copy every byte from disk to memory one at a time. DMA lets devices transfer data directly to/from RAM without CPU involvement. The CPU sets up the transfer (source, destination, size), the DMA controller does the work, and interrupts the CPU when done. This frees the CPU for other tasks.

Buses and I/O

Components communicate via **buses** — shared electrical pathways. The **memory bus** connects CPU to RAM. The **I/O bus** (PCIe on modern systems) connects to devices like GPUs, SSDs, and network cards. The OS uses **device drivers** — software modules that know how to talk to specific hardware.

Chapter 3

Processes and Process Management

What Is a Process?

A **process** is a program in execution. A program is a static file on disk (like /usr/bin/python3). When you run it, the OS creates a process: it allocates memory, loads the program code, sets up a stack, and starts executing. Each process is independent — it has its own memory space, and one process crashing does not (usually) crash others.

Process Memory Layout

```
High Address  ┌──────────────────────┐
              │      Stack       │ ← grows downward (local vars, return addrs)
              │        ↓         │
              │                  │
              │        ↑         │
              │      Heap        │ ← grows upward (malloc, new)
              ├──────────────────────┤
              │   BSS (uninit)   │ ← uninitialized global vars (zeroed)
              │   Data (init)    │ ← initialized global vars
              │   Text (code)    │ ← program instructions (read-only)
Low Address   └──────────────────────┘
```

Process States

Every process is in one of these states:

State	Meaning	Transition
New	Being created	→ Ready (when loaded)
Ready	Waiting for CPU	→ Running (when scheduled)
Running	Executing on CPU	→ Ready (preempted) or Waiting (I/O)
Waiting/Blocked	Waiting for I/O or event	→ Ready (when I/O completes)
Terminated	Finished execution	Resources reclaimed by OS

Process Control Block (PCB)

The OS stores all information about a process in a data structure called the **PCB** (or task_struct in Linux). It contains: process ID (PID), process state, program counter, CPU registers, memory management info (page tables), I/O status, scheduling info (priority, time used), and parent/child relationships.

```
// Simplified PCB structure
struct pcb {
    int pid;              // Unique process ID
    int state;            // RUNNING, READY, BLOCKED, etc.
    int priority;         // Scheduling priority
    unsigned long pc;     // Program counter
    unsigned long sp;     // Stack pointer
    unsigned long regs[16]; // Saved CPU registers
    struct page_table *pgdir; // Virtual memory page table
    int parent_pid;       // Parent process ID
    struct file *open_files[256]; // Open file descriptors
};
```

Context Switch

When the OS switches from running Process A to Process B, it performs a **context switch**: (1) Save A's registers and PC into A's PCB. (2) Load B's registers and PC from B's PCB. (3) Switch the page table to B's address space. (4) Resume B. Context switches cost 1-10 microseconds and are pure overhead — no useful work is done during the switch.

Process Creation

On Unix/Linux, new processes are created with **fork()**, which clones the calling process. The child gets a copy of the parent's memory (using copy-on-write for efficiency). The child then typically calls **exec()** to replace its memory with a new program. On Windows, **CreateProcess()** combines both steps.

```
// Unix process creation
#include <unistd.h>
#include <stdio.h>
int main() {
    pid_t pid = fork();  // Create child process
    if (pid == 0) {
        // Child process
        printf("Child: PID = %d\n", getpid());
        execl("/bin/ls", "ls", "-l", NULL); // Replace with ls
    } else if (pid > 0) {
        // Parent process
        printf("Parent: child PID = %d\n", pid);
        wait(NULL);  // Wait for child to finish
    } else {
        perror("fork failed");
    }
    return 0;
}
```

Chapter 4

Threads and Concurrency Basics

What Is a Thread?

A **thread** is the smallest unit of execution within a process. A process starts with one thread (the main thread) and can create more. All threads in a process share the same memory space (code, data, heap) but each thread has its own stack and registers.

Aspect	Process	Thread
Memory	Own address space	Shared with other threads in process
Creation cost	Expensive (copy page tables)	Cheap (share existing space)
Communication	IPC needed (pipes, sockets)	Direct shared memory access
Crash isolation	One crash does not affect others	One crash kills all threads in process
Context switch	Expensive (flush TLB, switch page table)	Cheap (same address space)

Why Use Threads?

- **Responsiveness**: A web browser uses one thread per tab. If one tab loads slowly, others remain responsive.
- **Resource sharing**: Threads share memory, so passing data between them is trivial (just use a shared variable).
- **Performance**: On a 4-core CPU, 4 threads can run truly in parallel, giving up to 4x speedup.
- **Economy**: Creating a thread is 10-100x cheaper than creating a process.

User-Level vs Kernel-Level Threads

User-level threads: Managed by a library in user space. The kernel sees only one thread per process. Fast to create/switch, but if one thread blocks on I/O, all threads in that process block. **Kernel-level threads**: Managed by the OS kernel. Each thread can be scheduled independently. If one thread blocks, others continue. This is what modern OSes (Linux, Windows, macOS) use. The mapping is called the **threading model**: one-to-one (each user thread maps to one kernel thread — used by Linux/Windows), many-to-one (multiple user threads map to one kernel thread — green threads), many-to-many (M user threads map to N kernel threads).

Creating Threads (POSIX pthreads)

```
#include <pthread.h>
#include <stdio.h>
void *print_hello(void *arg) {
    int id = *(int *)arg;
    printf("Hello from thread %d\n", id);
    return NULL;
}
int main() {
    pthread_t threads[4];
    int ids[4];
    for (int i = 0; i < 4; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, print_hello, &ids[i]);
    }
    for (int i = 0; i < 4; i++) {
        pthread_join(threads[i], NULL);  // Wait for each thread
```

```
    }
    return 0;
}
```

The Race Condition Problem

When two threads access shared data concurrently, and at least one writes, the result depends on timing — this is a **race condition**. Example: two threads incrementing a shared counter. If counter = 5, both read 5, both write 6. The counter should be 7 but ends up as 6. This is a bug. Synchronization (Chapter 8) is the solution.

Chapter 5

CPU Scheduling Algorithms

The Scheduling Problem

Multiple processes are ready to run, but there is typically only one CPU (or a few cores). The **scheduler** decides which process runs next, for how long, and in what order. Good scheduling maximizes CPU utilization, minimizes response time, and is fair.

Key Metrics

Metric	Definition	Goal
CPU Utilization	% of time CPU is doing useful work	Maximize (ideally 40-90%)
Throughput	Number of processes completed per unit time	Maximize
Turnaround Time	Total time from submission to completion	Minimize
Waiting Time	Total time spent in the ready queue	Minimize
Response Time	Time from submission to first response	Minimize (for interactive)

First Come First Served (FCFS)

Simplest: processes run in arrival order. Non-preemptive. Problem: **convoy effect** — a long CPU-bound process delays all shorter processes behind it. Average waiting time can be very high.

```
// FCFS Example:
// Process  Burst Time  Arrival
// P1       24ms        0
// P2        3ms        0
// P3        3ms        0
// Order: P1 -> P2 -> P3
// Waiting: P1=0, P2=24, P3=27. Average = 17ms
// If order were P2,P3,P1: Average = (0+3+6)/3 = 3ms!
```

Shortest Job First (SJF)

Run the process with the shortest burst time first. Provably optimal for minimizing average waiting time. Problem: requires knowing burst times in advance (impossible in practice — must estimate). Also causes **starvation**: long processes may never run if short ones keep arriving.

Round Robin (RR)

Each process gets a fixed **time quantum** (e.g., 10-100ms). After the quantum expires, the process is preempted and placed at the back of the ready queue. The next process runs. Fair and responsive, but choosing the right quantum is critical: too small → excessive context switches; too large → degrades to FCFS.

```
// Round Robin Example (quantum = 4ms):
// Process  Burst
// P1       10ms
// P2        4ms
// P3        6ms
// Timeline: [P1:0-4][P2:4-8][P3:8-12][P1:12-16][P3:16-18][P1:18-20]
// All processes get regular CPU time — responsive!
```

Priority Scheduling

Each process has a priority number. The highest-priority process runs first. Can be preemptive or non-preemptive. Problem: **starvation** — low-priority processes may never run. Solution: **aging** — gradually increase priority of waiting processes.

Multilevel Feedback Queue (MLFQ)

Uses multiple queues with different priorities and time quanta. New processes start in the highest-priority queue with a small quantum. If a process uses its entire quantum (CPU-bound), it drops to a lower queue with a larger quantum. I/O-bound processes stay high because they voluntarily yield before quantum expires. Periodically, all processes are boosted back to the top queue (prevents starvation). This is the basis for most real-world schedulers.

Chapter 6

Memory Management Fundamentals

Why Memory Management?

Every running process needs memory for its code, data, stack, and heap. The OS must: (1) Allocate memory to processes. (2) Protect processes from each other. (3) Manage the limited physical RAM efficiently. (4) Provide the illusion that each process has its own large, contiguous memory space.

Address Binding

Programs use memory addresses. But when should addresses be determined? **Compile time**: Absolute addresses baked in (only works if program always loads at same address). **Load time**: Addresses computed when program is loaded. **Execution time**: Addresses translated dynamically via hardware (MMU) — this is what modern OSes use, enabling virtual memory.

Contiguous Allocation

Simplest approach: give each process a contiguous block of memory. The OS maintains a list of free memory holes. When a process needs memory, find a hole. Strategies:

Strategy	How It Works	Pros/Cons
First Fit	Allocate first hole big enough	Fast, but wastes upper memory
Best Fit	Allocate smallest hole big enough	Less waste, but slow (scan all holes)
Worst Fit	Allocate largest hole	Leaves biggest remaining hole, but slow

Problem: **External fragmentation** — free memory is split into small, unusable chunks. Solution: **compaction** (move processes to eliminate gaps — expensive) or **paging** (Chapter 7).

Segmentation

Divide a process's memory into logical **segments**: code, data, stack, heap. Each segment has a base address and limit. The hardware checks every memory access: if offset >= limit, trap (segfault). This maps naturally to how programmers think about memory, but still suffers from external fragmentation (segments are variable-size).

Paging (Introduction)

Divide physical memory into fixed-size **frames** (typically 4 KB). Divide each process's virtual memory into same-size **pages**. A **page table** maps each page to a frame. No external fragmentation (all chunks are same size). Small internal fragmentation (last page may be partially empty). This is the foundation of modern memory management — explored in depth in Chapter 7.

```
// Virtual address translation:
// Virtual address: [Page Number | Offset]
// Page table maps: Page Number -> Frame Number
// Physical address: [Frame Number | Offset]
// Example: 4 KB pages, 32-bit address space
// Page size = 4096 = 2^12, so offset = 12 bits
// Page number = 32 - 12 = 20 bits = 1M pages
// Virtual addr 0x00005A3C:
//   Page number = 0x00005 (page 5)
//   Offset = 0xA3C
//   If page 5 maps to frame 100:
```

```
//   Physical addr = 100 * 4096 + 0xA3C = 0x64A3C
```

Chapter 7

Virtual Memory

The Big Idea

Virtual memory gives each process the illusion of having a large, contiguous, private address space — even if physical RAM is much smaller. The OS and hardware (MMU) collaborate to map virtual addresses to physical addresses. Pages that are not currently needed can live on disk and be loaded on demand.

Demand Paging

Instead of loading an entire program into memory, only load pages as they are accessed. When a process accesses a page not in RAM, a **page fault** occurs: (1) CPU raises an exception. (2) OS checks if the access is valid (is this a legal address?). (3) If valid, OS finds a free frame, loads the page from disk, updates the page table, and restarts the instruction. If all frames are full, the OS must evict a page first (Chapter 13).

Page Table Entry (PTE)

```
// A page table entry contains:
struct pte {
    unsigned valid    : 1;  // 1 = page is in RAM
    unsigned write    : 1;  // 1 = writable
    unsigned user     : 1;  // 1 = user-accessible
    unsigned accessed : 1;  // set by hardware on access
    unsigned dirty    : 1;  // set by hardware on write
    unsigned frame    : 20; // physical frame number
    // (simplified for 32-bit, 4 KB pages)
};
```

Multi-Level Page Tables

A flat page table for a 32-bit address space needs 1M entries × 4 bytes = 4 MB per process. For 64-bit, it would be impossibly large. Solution: hierarchical page tables. x86-64 uses **4-level page tables**: PML4 → PDPT → PD → PT → Frame. Only the portions of the table that map allocated memory actually exist in RAM, saving enormous amounts of memory.

```
// x86-64 virtual address breakdown (48-bit virtual):
// [PML4 idx (9 bits)][PDPT idx (9 bits)][PD idx (9 bits)]
// [PT idx (9 bits)][Offset (12 bits)]
// Translation: CR3 -> PML4[idx] -> PDPT[idx] -> PD[idx]
//              -> PT[idx] -> Physical frame + offset
// 4 memory accesses just to translate one address!
```

TLB (Translation Lookaside Buffer)

Walking the page table for every memory access would be painfully slow (4 extra memory accesses on x86-64). The **TLB** is a small, fast cache (typically 64-1024 entries) that stores recent virtual-to-physical translations. TLB hit: ~1 cycle. TLB miss: ~100+ cycles (page table walk). TLB hit rates are typically 99%+ for well-behaving programs due to **locality of reference**: programs tend to access the same pages repeatedly.

Copy-on-Write (COW)

When fork() creates a child process, it does not copy all the parent's memory. Instead, both parent and child share the same physical frames, marked read-only. When either process tries to write, the hardware traps, and the OS copies just

that page. This makes fork() nearly instantaneous, even for large processes.

Thrashing

If a process does not have enough frames, it page faults constantly — loading one page evicts another that will be needed soon. CPU utilization drops to near zero because the system spends all its time swapping pages. This is **thrashing**. Solutions: give processes more frames, use working set model, or reduce multiprogramming degree.

Chapter 8

Synchronization and Mutual Exclusion

The Critical Section Problem

When multiple threads access shared data, we need to ensure that only one thread accesses the shared data at a time. The code that accesses shared data is called the **critical section**. A solution must satisfy three properties:

- **Mutual exclusion**: Only one thread in the critical section at a time.
- **Progress**: If no thread is in the critical section, a waiting thread can enter.
- **Bounded waiting**: A thread cannot be forced to wait forever.

Mutex (Mutual Exclusion Lock)

The simplest synchronization primitive. A mutex has two states: locked and unlocked. Only one thread can hold the lock at a time. Other threads that try to lock it will block (sleep) until it is released.

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int shared_counter = 0;
void *increment(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&lock);    // Enter critical section
        shared_counter++;             // Safe: only one thread here
        pthread_mutex_unlock(&lock);  // Leave critical section
    }
    return NULL;
}
// Without the mutex, two threads running this would lose updates.
// With the mutex, shared_counter correctly reaches 2000000.
```

Semaphores

A **semaphore** is a generalized mutex with a counter. **wait()** (P operation) decrements the counter; if it becomes negative, the thread blocks. **signal()** (V operation) increments the counter; if threads are waiting, one is woken up. A semaphore initialized to 1 behaves like a mutex. Initialized to N, it allows up to N threads into the critical section (useful for limiting concurrent access to a resource pool like database connections).

Condition Variables

Sometimes a thread needs to wait until a specific condition is true (e.g., buffer is not empty). A **condition variable** lets a thread sleep until another thread signals the condition. Always used with a mutex. Pattern: lock mutex → check condition → if false, wait on condvar (releases mutex and sleeps) → when signaled, re-acquire mutex and recheck condition.

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int data_ready = 0;
// Producer
pthread_mutex_lock(&mutex);
data_ready = 1;
pthread_cond_signal(&cond);  // Wake one waiting thread
pthread_mutex_unlock(&mutex);
// Consumer
pthread_mutex_lock(&mutex);
while (!data_ready) {         // Always use while, not if
    pthread_cond_wait(&cond, &mutex); // Atomically unlock+sleep
```

```
}
// Process data...
pthread_mutex_unlock(&mutex);
```

Always use while(!condition) with condition variables, not if(!condition). Spurious wakeups can occur.

Chapter 9

Deadlocks

What Is a Deadlock?

A **deadlock** occurs when two or more threads are each waiting for a resource held by another, forming a circular wait. None can proceed. Example: Thread A holds Lock 1 and waits for Lock 2. Thread B holds Lock 2 and waits for Lock 1. Neither can continue.

Four Necessary Conditions (Coffman Conditions)

All four must hold simultaneously for a deadlock to exist:

Condition	Meaning	How to Break It
Mutual Exclusion	Resources are non-sharable	Use sharable resources when possible
Hold and Wait	Hold one resource, wait for another	Request all resources at once
No Preemption	Resources cannot be forcibly taken	Allow OS to preempt resources
Circular Wait	Circular chain of waiting	Impose ordering on resource requests

Deadlock Prevention

Break one of the four conditions. The most practical: **impose a total ordering** on resources and require that threads always acquire resources in that order. If Lock 1 < Lock 2, every thread must acquire Lock 1 before Lock 2. This prevents circular wait.

```
// DEADLOCK-PRONE (different lock ordering):
// Thread A: lock(mutex1); lock(mutex2);
// Thread B: lock(mutex2); lock(mutex1);  // DANGER!
// DEADLOCK-FREE (consistent ordering):
// Thread A: lock(mutex1); lock(mutex2);
// Thread B: lock(mutex1); lock(mutex2);  // Same order = safe
```

Deadlock Avoidance: Banker's Algorithm

Before granting a resource, check if the system will remain in a **safe state** — a state where there exists at least one sequence in which all processes can finish. The Banker's algorithm checks this by simulating whether remaining resources can satisfy at least one process, freeing its resources, and repeating. If granting the request leads to an unsafe state, the request is denied and the thread waits.

Deadlock Detection and Recovery

Allow deadlocks to happen, then detect and fix them. Detection: build a wait-for graph (nodes = threads, edge A→B means A waits for B). A cycle = deadlock. Recovery options: (1) Kill one of the deadlocked threads. (2) Preempt a resource from one thread. (3) Rollback a thread to a previous checkpoint. Most production systems use a combination of prevention (lock ordering) and detection (timeouts).

Chapter 10

File Systems and I/O Basics

What Is a File System?

A **file system** organizes data on a storage device (HDD, SSD) into files and directories. It provides: naming (human-readable names instead of raw block numbers), persistence (data survives reboots), organization (hierarchical directory structure), and access control (permissions).

File Operations

Operation	System Call (Unix)	Description
Create	open(path, O_CREAT)	Create new file
Open	open(path, flags)	Get file descriptor
Read	read(fd, buf, n)	Read n bytes into buffer
Write	write(fd, buf, n)	Write n bytes from buffer
Seek	lseek(fd, offset, whence)	Move read/write position
Close	close(fd)	Release file descriptor
Delete	unlink(path)	Remove file (when link count = 0)

Directories

A **directory** is a special file that contains a list of (name, inode number) pairs. The root directory is /. Path resolution: /home/user/file.txt means: look up 'home' in / (get inode), look up 'user' in /home (get inode), look up 'file.txt' in /home/user (get inode). Each step requires reading the directory's data blocks.

Inodes

On Unix file systems, each file has an **inode** (index node) that stores metadata: file type, permissions, owner, size, timestamps, and pointers to data blocks. The file name is NOT stored in the inode — it is stored in the directory entry. This is why hard links work: multiple directory entries can point to the same inode.

Disk Layout

```
// Typical disk layout (ext4-like):
[ Boot Block | Superblock | Inode Bitmap | Data Bitmap
  | Inode Table | Data Blocks ]
// Boot Block: loaded by BIOS/UEFI to start OS
// Superblock: filesystem metadata (size, block size, counts)
// Inode Bitmap: 1 bit per inode (free/used)
// Data Bitmap: 1 bit per data block (free/used)
// Inode Table: array of inode structs
// Data Blocks: actual file contents
```

I/O Basics

Programs access devices through **device drivers** — kernel modules that know how to talk to specific hardware. The OS provides a uniform interface: everything is a file (on Unix). Reading from a keyboard, a disk, or a network socket all use the same read() system call. I/O can be **blocking** (thread waits until complete), **non-blocking** (returns immediately if not

ready), or **asynchronous** (OS notifies when complete).

Buffering and Caching

The OS maintains a **buffer cache** (also called page cache) — a region of RAM that stores recently accessed disk blocks. When you read a file, the OS first checks the cache. If found (cache hit), no disk I/O needed. Writes can be buffered too: the OS writes to cache and flushes to disk later (write-back). This dramatically improves performance since RAM is ~100,000x faster than disk.

PART II

INTERMEDIATE

For practitioners deepening their skills

Chapter 11

Advanced Process Management

Process Relationships

On Unix, processes form a tree. Process 1 (**init** or **systemd**) is the root. Every process has a parent (except init). When a process calls fork(), the child's parent is the caller. When a parent terminates before its child, the child becomes an **orphan** and is re-parented to init. When a child terminates but the parent has not called wait(), the child becomes a **zombie** — its PCB remains in the process table until the parent collects the exit status.

Process Groups and Sessions

Processes are organized into **process groups** for signal delivery (e.g., Ctrl+C sends SIGINT to the entire foreground process group). A **session** is a collection of process groups, typically associated with a terminal. The **session leader** is usually the login shell. Background jobs run in separate process groups.

Signals

Signals are software interrupts sent to processes. Common signals:

Signal	Number	Default Action	Use
SIGINT	2	Terminate	Ctrl+C — interrupt process
SIGKILL	9	Terminate (cannot be caught)	Force kill unresponsive process
SIGSEGV	11	Core dump	Segmentation fault (invalid memory access)
SIGTERM	15	Terminate	Polite termination request
SIGSTOP	19	Stop (cannot be caught)	Pause process
SIGCONT	18	Continue	Resume stopped process
SIGCHLD	17	Ignore	Child process terminated

Process Scheduling Policies in Linux

Linux supports multiple scheduling policies: **SCHED_OTHER** (default, CFS-based, for normal processes), **SCHED_FIFO** (real-time, runs until voluntarily yields or higher-priority preempts), **SCHED_RR** (real-time round-robin with time quantum), and **SCHED_DEADLINE** (earliest deadline first for periodic real-time tasks). Real-time processes always preempt normal processes.

Chapter 12

Advanced Scheduling: CFS, MLFQ, Real-Time

Completely Fair Scheduler (CFS)

Linux's default scheduler since kernel 2.6.23 (2007). Key idea: track each task's **virtual runtime** (vruntime) — the total CPU time weighted by priority. The task with the smallest vruntime runs next. Tasks are stored in a red-black tree sorted by vruntime. Lookup and insertion are O(log n). High-priority tasks accumulate vruntime slower, so they run more often.

```
// CFS virtual runtime calculation:
// vruntime += delta_exec * (NICE_0_WEIGHT / task_weight)
// NICE_0_WEIGHT = 1024 (nice 0)
// nice -20: weight = 88761 -> vruntime grows ~87x slower
// nice  19: weight = 15    -> vruntime grows ~68x faster
// Result: nice -20 task gets ~87x more CPU than nice 19 task
```

MLFQ in Practice

The Multilevel Feedback Queue uses heuristics to distinguish CPU-bound from I/O-bound processes: (1) Start all new processes at highest priority. (2) If a process uses its full quantum, demote it (likely CPU-bound). (3) If it yields before quantum expires, keep it high (likely I/O-bound). (4) Periodically boost all processes (prevents starvation). FreeBSD's ULE scheduler and Windows' dispatcher use MLFQ variants.

Real-Time Scheduling

Hard real-time: Missing a deadline is a system failure (aircraft control, pacemakers). **Soft real-time**: Missing a deadline degrades quality but is not catastrophic (video playback, gaming). Algorithms:

Algorithm	How It Works	Optimal?
Rate Monotonic (RM)	Static priority by period (shorter period = higher)	Optimal among fixed-priority
Earliest Deadline First (EDF)	Dynamic priority by deadline (closest deadline first)	Optimal if preemptive + uniprocessor
SCHED_DEADLINE (Linux)	CBS + EDF implementation in Linux kernel	Practical EDF for Linux

Multiprocessor Scheduling

With multiple CPUs: **Symmetric multiprocessing (SMP)** — each CPU runs the scheduler independently. **Load balancing** migrates tasks between CPUs. **Processor affinity** tries to keep a task on the same CPU (to preserve cache warmth). Linux uses per-CPU run queues with periodic load balancing.

Chapter 13

Page Replacement Algorithms

The Problem

When physical RAM is full and a new page must be loaded, the OS must choose a **victim page** to evict. If the victim has been modified (dirty), it must be written to disk first. The goal: minimize page faults.

Algorithm Comparison

Algorithm	Strategy	Practical?	Performance
OPT (Belady's)	Evict page used farthest in future	No (needs future knowledge)	Best possible (baseline)
FIFO	Evict oldest page	Yes	Poor (Belady's anomaly)
LRU	Evict least recently used page	Expensive (exact)	Near-optimal
Clock (Second Chance)	Circular scan with reference bit	Yes (used in practice)	Good approximation of LRU
LFU	Evict least frequently used	Yes	Good for some workloads
Working Set	Keep recent pages in memory	Yes (adaptive)	Good (prevents thrashing)

The Clock Algorithm (Second Chance)

Used in most real OSes. Pages are arranged in a circular buffer. Each page has a **reference bit** (set to 1 by hardware on access). A clock hand sweeps around: if reference bit = 1, clear it and move on (give second chance). If reference bit = 0, evict this page. Simple, efficient, and approximates LRU well.

```
// Clock algorithm pseudocode:
function clock_replace():
    while true:
        page = pages[hand]
        if page.reference_bit == 0:
            evict(page)       // This page was not recently used
            return page.frame
        else:
            page.reference_bit = 0  // Give second chance
        hand = (hand + 1) % num_frames  // Advance clock hand
```

Working Set Model

The **working set** of a process at time t is the set of pages referenced in the last Δ time units. If a process has fewer frames than its working set size, it will thrash. The OS can monitor working set sizes and only admit processes that fit. If total working set sizes exceed physical memory, suspend a process.

Chapter 14

Advanced Synchronization Primitives

Spinlocks

A **spinlock** busy-waits (loops) until the lock is available. Unlike mutexes (which put the thread to sleep), spinlocks waste CPU cycles while waiting. Use spinlocks only when: (1) hold time is very short (< a few μs), and (2) sleeping is not an option (e.g., in interrupt handlers). In the kernel, spinlocks are common because the scheduler itself uses them.

```
// Spinlock using atomic test-and-set:
void spin_lock(int *lock) {
    while (__sync_lock_test_and_set(lock, 1)) {
        // Spin: CPU burns cycles here
        while (*lock) { }  // spin on read (cache-friendly)
    }
}
void spin_unlock(int *lock) {
    __sync_lock_release(lock);
}
```

Read-Write Locks

Allow multiple readers OR one writer. Useful for data read far more often than written (e.g., configuration, routing tables). Readers do not block each other, only writers are exclusive. Variants: reader-preference (may starve writers), writer-preference (may starve readers), fair (alternating).

Monitors

A **monitor** is a high-level synchronization construct (used in Java's synchronized keyword). It bundles: (1) shared data, (2) operations on the data, and (3) mutual exclusion (only one thread executes a monitor procedure at a time). Condition variables are used inside monitors for waiting. Monitors are less error-prone than raw mutexes because the lock is acquired and released automatically.

Barriers

A **barrier** makes N threads wait until all N have arrived. Useful in parallel computing where all threads must complete phase 1 before starting phase 2.

Classic Problems

Problem	Description	Key Technique
Producer-Consumer	Producer adds to buffer, consumer removes	Mutex + 2 semaphores (empty/full)
Readers-Writers	Multiple readers or one writer	Read-write lock or mutex + counters
Dining Philosophers	5 philosophers, 5 forks, avoid deadlock	Resource ordering or semaphores

Chapter 15

File System Implementation

On-Disk Structures

A file system must organize data on disk to support efficient reading, writing, and metadata operations. The key structures: **Superblock** (filesystem metadata: size, block count, inode count), **Inode table** (array of inodes — one per file), **Data bitmap** and **Inode bitmap** (track free blocks/inodes), **Data blocks** (actual file content).

Inode Structure (ext4)

```
// ext4 inode (simplified):
struct ext4_inode {
    uint16_t mode;         // file type + permissions
    uint16_t uid;          // owner user ID
    uint32_t size_lo;      // file size (lower 32 bits)
    uint32_t atime;        // last access time
    uint32_t ctime;        // creation time
    uint32_t mtime;        // last modification time
    uint16_t links_count;  // hard link count
    uint32_t blocks;       // number of 512-byte blocks
    uint32_t block[15];    // block pointers:
    // [0-11]: 12 direct blocks = 48 KB
    // [12]: single indirect = 4 MB
    // [13]: double indirect = 4 GB
    // [14]: triple indirect = 4 TB
};
```

Journaling

If the system crashes mid-write, the file system can be left inconsistent. **Journaling** solves this: before making changes, write them to a log (journal). If crash occurs, replay the journal on recovery. Three modes:

Mode	What Is Journaled	Speed	Safety
Journal (full)	Metadata + data	Slow	Highest
Ordered (default ext4)	Metadata only, data written first	Medium	Good
Writeback	Metadata only, data in any order	Fast	Lower

Free Space Management

How does the OS track which blocks are free? **Bitmap**: one bit per block (1 = used, 0 = free). For a 1 TB disk with 4 KB blocks, the bitmap is 32 MB. **Linked list**: each free block points to the next. Space-efficient but slow for allocation. Most modern file systems use bitmaps with block groups for efficiency.

Chapter 16

I/O Systems and Device Drivers

I/O Hardware

The OS interacts with devices through **device controllers** — hardware chips on the device or motherboard. Each controller has registers: **status** (is device busy?), **command** (what to do), **data** (data to/from device). The OS accesses these via **port-mapped I/O** (special in/out instructions on x86) or **memory-mapped I/O** (device registers mapped to memory addresses).

I/O Methods

Method	How It Works	CPU Overhead
Programmed I/O (Polling)	CPU repeatedly checks device status	Very high (busy-waits)
Interrupt-driven I/O	Device interrupts CPU when ready	Medium (interrupt handling)
DMA (Direct Memory Access)	DMA controller transfers data, interrupts when done	Low (CPU free during transfer)

Device Driver Architecture

A **device driver** is a kernel module that translates generic OS I/O requests into device-specific commands. On Linux, drivers register with the kernel and provide function pointers for operations (open, read, write, ioctl, close). The kernel calls these functions when applications access the device.

Disk Scheduling

When multiple I/O requests are pending, the disk scheduler orders them to minimize seek time.

Algorithm	Strategy	Notes
FCFS	Process requests in order	Fair but slow
SSTF	Nearest request first	Efficient but can starve far requests
SCAN (Elevator)	Sweep back and forth across disk	Fair and efficient
C-SCAN	Sweep one direction, jump back	More uniform wait times
NOOP/None	No reordering	Best for SSDs (no seek time)

SSDs have no moving parts, so seek time is negligible. Disk scheduling matters primarily for HDDs.

Chapter 17

Inter-Process Communication (IPC)

Why IPC?

Processes have separate address spaces and cannot directly access each other's memory. IPC mechanisms allow processes to exchange data and coordinate. This is fundamental to modular software design — microservices, client-server architectures, and pipelines all rely on IPC.

IPC Mechanisms

Mechanism	Description	Speed	Scope
Pipes	Unidirectional byte stream (parent-child)	Fast	Same machine
Named Pipes (FIFO)	Pipe accessible by name in filesystem	Fast	Same machine
Message Queues	Structured messages with priorities	Medium	Same machine
Shared Memory	Processes map same physical memory	Fastest	Same machine
Sockets	Bidirectional communication (network-capable)	Medium	Network
Signals	Asynchronous notifications	Fast	Same machine
Memory-Mapped Files	File mapped into memory, shared	Fast	Same machine

Pipes

```
// Unix pipe example:
int fd[2];
pipe(fd);  // fd[0] = read end, fd[1] = write end
if (fork() == 0) {
    // Child: reads from pipe
    close(fd[1]);  // Close write end
    char buf[100];
    read(fd[0], buf, sizeof(buf));
    printf("Child received: %s\n", buf);
    close(fd[0]);
} else {
    // Parent: writes to pipe
    close(fd[0]);  // Close read end
    write(fd[1], "Hello from parent", 17);
    close(fd[1]);
    wait(NULL);
}
```

Shared Memory

Fastest IPC: both processes map the same physical memory region into their address spaces. No kernel involvement for reads/writes (just normal memory access). But you must synchronize access with semaphores or mutexes to avoid race conditions.

Chapter 18

Memory-Mapped Files and Shared Memory

Memory-Mapped I/O

The **mmap()** system call maps a file (or anonymous memory) directly into a process's virtual address space. Instead of using read()/write(), you access the file through regular pointer operations. The OS handles paging: if you access an unmapped region, a page fault loads it from disk.

```
// Memory-mapping a file in C:
#include <sys/mman.h>
int fd = open("data.bin", O_RDWR);
struct stat st;
fstat(fd, &st);
char *data = mmap(NULL, st.st_size, PROT_READ | PROT_WRITE,
                  MAP_SHARED, fd, 0);
// Now access data[0], data[1], etc. as regular memory
data[0] = 'H';  // Writes to file (eventually flushed)
munmap(data, st.st_size);
close(fd);
```

Benefits of mmap

- **Simplicity**: Treat files as arrays. No read()/write() buffering needed.
- **Performance**: Avoids copying between kernel buffer and user buffer. Zero-copy access.
- **Shared memory**: Two processes mapping the same file with MAP_SHARED see each other's changes.
- **Lazy loading**: Only pages actually accessed are loaded into RAM.

POSIX Shared Memory

For IPC without a file, use **shm_open()** + mmap(). Creates a shared memory object backed by RAM (not disk). Faster than file-backed mmap for pure IPC. Must synchronize access between processes using semaphores.

Chapter 19

Protection and Security Fundamentals

Protection vs Security

Protection controls which processes can access which resources (a mechanism). **Security** ensures the system is used only by authorized users in authorized ways (a policy). Protection implements security policies.

Access Control

Unix permission model: each file has owner, group, and others. Each has read (r), write (w), execute (x) permissions. Represented as 3 octal digits (e.g., 755 = rwxr-xr-x). This is a simple but effective **Discretionary Access Control (DAC)** model.

```
$ ls -l myfile.txt
-rw-r--r-- 1 alice staff 1024 Jan 1 12:00 myfile.txt
# Owner (alice): read+write
# Group (staff): read only
# Others: read only
$ chmod 750 myfile.txt  # rwxr-x---
```

Principle of Least Privilege

Every process should run with the minimum privileges necessary. Root (UID 0) can do everything — but running programs as root is dangerous. Modern systems use **capabilities** (Linux) to grant specific privileges (e.g., CAP_NET_BIND_SERVICE lets a process bind to ports < 1024 without full root).

Security Threats

Threat	Description	Defense
Buffer Overflow	Overwrite stack to hijack control flow	Stack canaries, ASLR, NX bit
Privilege Escalation	Gain higher privileges than authorized	Principle of least privilege, SUID audit
Malware	Malicious software (virus, worm, trojan)	Antivirus, sandboxing, code signing
Denial of Service	Overwhelm system resources	Rate limiting, resource quotas

Chapter 20

Introduction to Kernel Architecture

What Is a Kernel?

The **kernel** is the core of the OS — the code that runs with full hardware privileges. It manages processes, memory, devices, file systems, and networking. Everything else (applications, utilities, shells) runs in user space with restricted privileges.

Kernel Architecture Types

Type	Description	Examples	Trade-offs
Monolithic	Entire OS in one big program in kernel space	Linux, FreeBSD	Fast (no IPC), but large attack surface
Microkernel	Minimal kernel; services in user space	QNX, MINIX 3, seL4	Secure, modular, but slower (IPC overhead)
Hybrid	Monolithic core + some services in user space	Windows NT, macOS (XNU)	Balanced performance and modularity
Exokernel	Minimal kernel; apps manage own resources	MIT Exokernel (research)	Maximum flexibility, complex to program

System Call Interface

Applications request kernel services via **system calls**. On x86-64 Linux: the application places the syscall number in register rax, arguments in rdi, rsi, rdx, etc., then executes the **syscall** instruction. This triggers a mode switch to kernel mode, where the kernel's syscall handler dispatches to the appropriate function. After completion, the kernel returns the result in rax and switches back to user mode.

Loadable Kernel Modules (LKMs)

Linux supports **loadable kernel modules** — code that can be loaded into the running kernel without rebooting. Most device drivers are LKMs. This gives the modularity of a microkernel with the performance of a monolithic kernel.

```
// Basic Linux kernel module:
#include <linux/module.h>
#include <linux/kernel.h>
static int __init hello_init(void) {
    printk(KERN_INFO "Hello, kernel!\n");
    return 0;
}
static void __exit hello_exit(void) {
    printk(KERN_INFO "Goodbye, kernel!\n");
}
module_init(hello_init);
module_exit(hello_exit);
MODULE_LICENSE("GPL");
```

PART III

ADVANCED

For those seeking mastery of OS internals

Chapter 21

Kernel Internals: Monolithic vs Microkernel

Monolithic Kernel Deep Dive

In a monolithic kernel (Linux, FreeBSD), all OS services run in a single address space in kernel mode: process scheduler, memory manager, file systems, network stack, device drivers. Advantages: fast (function calls instead of IPC), easy data sharing. Disadvantages: a bug in any component can crash the entire system; large code base (Linux kernel: ~30 million lines).

Microkernel Design

A microkernel keeps only the absolute minimum in kernel space: address space management, thread scheduling, and IPC. Everything else (file systems, drivers, network stack) runs as user-space **servers**. Communication is via message passing. This is more secure and fault-tolerant (a crashed driver does not crash the kernel) but slower (every OS service call requires context switches and IPC). seL4 is a formally verified microkernel — mathematically proven to be correct.

The IPC Overhead Problem

The main criticism of microkernels is IPC overhead. A file read in a monolithic kernel: 1 context switch. In a microkernel: user → kernel → file server (user) → kernel → disk driver (user) → kernel → back. That is 6 context switches vs 2. L4 microkernels have reduced this overhead to ~1 μs per IPC, making it practical.

Hybrid Kernels

Windows NT and macOS/XNU take a middle path. The kernel includes essential services (scheduling, memory, some drivers) but also supports user-space components. Windows NT has a Hardware Abstraction Layer (HAL), Executive (object manager, I/O manager, etc.), and a microkernel core. XNU combines a Mach microkernel with BSD Unix monolithic code.

Chapter 22

Lock-Free and Wait-Free Data Structures

Why Lock-Free?

Mutexes have problems: priority inversion, convoying, and they don't compose (holding lock A while calling code that takes lock B risks deadlock). **Lock-free** algorithms guarantee that at least one thread makes progress (no deadlock). **Wait-free** algorithms guarantee every thread makes progress in bounded steps (no starvation).

Compare-And-Swap (CAS)

The foundation of lock-free programming. **CAS(addr, expected, new_val)**: atomically, if *addr == expected, set *addr = new_val and return true; else return false. Supported by hardware: x86 CMPXCHG, ARM LDREX/STREX.

```
// Lock-free stack (Treiber stack):
struct node { int val; struct node *next; };
struct node *top = NULL;  // atomic pointer
void push(int val) {
    struct node *n = malloc(sizeof(*n));
    n->val = val;
    do {
        n->next = top;             // Read current top
    } while (!CAS(&top, n->next, n)); // Swing top to new node
}
int pop() {
    struct node *old_top;
    do {
        old_top = top;
        if (!old_top) return -1;   // Stack empty
    } while (!CAS(&top, old_top, old_top->next));
    int val = old_top->val;
    free(old_top);  // Careful: ABA problem!
    return val;
}
```

The ABA Problem

Thread 1 reads A from a pointer. Thread 2 changes it to B then back to A. Thread 1's CAS succeeds (still A), but the data may have changed. Solutions: tagged pointers (counter + pointer), hazard pointers, or epoch-based reclamation (used in Linux RCU).

Memory Ordering

Modern CPUs reorder memory operations for performance. This can break lock-free algorithms. **Memory barriers** (fences) enforce ordering. C11/C++11 provide memory order options: seq_cst (strongest, default), acquire/release (sufficient for most patterns), relaxed (no ordering guarantee). Using the wrong memory order causes subtle, hard-to-reproduce bugs.

Chapter 23

Advanced Virtual Memory: TLB, Huge Pages, NUMA

TLB Internals

The TLB is split into: **ITLB** (instruction fetches) and **DTLB** (data accesses). Typical sizes: L1 DTLB: 64 entries (4 KB pages) + 32 entries (2 MB pages). L2 TLB: 1536 entries (shared). TLB miss penalty on x86-64: hardware page table walker performs 4 memory accesses (PML4 → PGD → PMD → PTE). TLB shootdown: when one CPU modifies a page table, all other CPUs' TLBs must invalidate the affected entries — this is expensive on multicore systems.

Huge Pages

Standard 4 KB pages mean a 1 GB dataset spans 262,144 pages = 262,144 TLB entries needed. With **2 MB huge pages**: only 512 entries. With **1 GB huge pages**: just 1 entry. Huge pages dramatically reduce TLB misses for large datasets (databases, VMs). Linux supports: explicit huge pages (hugetlbfs, application must request), and Transparent Huge Pages (THP) — the kernel automatically promotes to 2 MB when it can.

NUMA (Non-Uniform Memory Access)

In multi-socket servers, each CPU socket has its own local memory. Accessing local memory: ~100 ns. Accessing remote memory (other socket): ~300 ns. The OS must be NUMA-aware: allocate memory on the same node as the CPU running the process. Linux uses a NUMA first-touch policy by default: memory is allocated on the node where it is first accessed. Tools: numactl, libnuma, /sys/devices/system/node/.

Concept	4 KB Pages	2 MB Huge Pages	1 GB Huge Pages
TLB entries for 1 GB	262,144	512	1
TLB miss rate	High for large data	Low	Minimal
Internal fragmentation	Low (0-4 KB)	Up to 2 MB wasted	Up to 1 GB wasted
Use case	General purpose	Databases, VMs	Very large datasets

Chapter 24

Advanced File Systems: ext4, ZFS, Btrfs

ext4

The default Linux file system. Key features: **extents** (contiguous block ranges replace individual block pointers — more efficient for large files), **delayed allocation** (allocate blocks at flush time, not write time — better locality), **journal checksumming**, max file size 16 TB, max volume 1 EB. Limitations: no built-in snapshots, no checksumming of data, no RAID.

ZFS

Developed by Sun Microsystems. A combined file system + volume manager. Key features: **copy-on-write** (never overwrites existing data), **end-to-end checksumming** (detects silent data corruption), **snapshots and clones** (instant, zero-cost), **built-in RAID-Z** (RAID 5/6 without the write hole), **compression** (transparent LZ4/zstd), **deduplication**. Widely used in enterprise storage, NAS (FreeNAS/TrueNAS). RAM-hungry: wants 1 GB RAM per TB of storage for dedup.

Btrfs

Linux's modern COW file system. Features similar to ZFS: snapshots, checksumming, built-in RAID, compression, subvolumes. Advantage: in mainline Linux kernel (ZFS is not, due to licensing). Disadvantage: RAID 5/6 implementation still has known issues. Used by SUSE, Facebook (for some workloads), and Synology NAS.

Comparison

Feature	ext4	ZFS	Btrfs
Copy-on-write	No	Yes	Yes
Checksumming	Metadata only	Metadata + Data	Metadata + Data
Snapshots	No	Yes (instant)	Yes (instant)
Built-in RAID	No	Yes (RAID-Z)	Yes (but RAID5/6 unstable)
Max file size	16 TB	16 EB	16 EB
Compression	No	Yes (LZ4, zstd)	Yes (zlib, lzo, zstd)
Maturity	Very mature	Very mature	Maturing

Chapter 25

Virtualization and Hypervisors

What Is Virtualization?

Virtualization creates **virtual machines (VMs)** — software abstractions that behave like physical computers. Each VM runs its own OS (guest OS) on top of a **hypervisor** that manages hardware sharing.

Hypervisor Types

Type	Description	Examples
Type 1 (Bare-metal)	Runs directly on hardware, no host OS	VMware ESXi, Xen, KVM, Hyper-V
Type 2 (Hosted)	Runs on top of a host OS	VirtualBox, VMware Workstation

Hardware-Assisted Virtualization

Modern CPUs have virtualization extensions: Intel VT-x, AMD-V. These add a new privilege level (VMX root/non-root). The hypervisor runs in VMX root mode, guests in VMX non-root mode. Sensitive instructions by the guest automatically trap to the hypervisor (VM exit), which emulates them. This eliminates the need for binary translation (used before hardware support).

Memory Virtualization

Guest OS has its own page tables (virtual → guest physical). Hypervisor has another layer (guest physical → host physical). **Extended Page Tables (EPT)** on Intel (or **NPT** on AMD) let the hardware walk both levels in one step, avoiding costly shadow page tables.

KVM (Kernel-based Virtual Machine)

Linux's built-in hypervisor. KVM turns the Linux kernel into a Type 1 hypervisor. Each VM is a regular Linux process. KVM uses VT-x/AMD-V for CPU virtualization, EPT/NPT for memory, and QEMU for device emulation. Virtio provides paravirtualized I/O devices for better performance.

Chapter 26

Containers and OS-Level Virtualization

Containers vs VMs

Containers are **OS-level virtualization** — they share the host kernel but have isolated file systems, network, and process spaces. Much lighter than VMs (no guest OS overhead).

Aspect	Virtual Machines	Containers
Isolation	Strong (separate kernel)	Moderate (shared kernel)
Startup time	Minutes	Seconds
Memory overhead	GBs (full OS)	MBs (just app + libs)
Performance	Near-native (with HW assist)	Native (no hypervisor layer)
Use case	Running different OSes, strong isolation	Microservices, CI/CD, development

Linux Building Blocks

Containers use two key kernel features: **Namespaces** isolate what a process can see (PID, network, mount, user, IPC, UTS, cgroup, time). A process in a PID namespace sees itself as PID 1. **Cgroups** (control groups) limit resources: CPU shares, memory limits, I/O bandwidth, number of processes. Together, namespaces + cgroups = container.

Docker and OCI

Docker popularized containers by making them easy to use. A Docker image bundles the application + dependencies. Docker uses containerd as its runtime, which uses runc to create containers via namespaces + cgroups. The Open Container Initiative (OCI) standardizes container image format and runtime specification.

Chapter 27

Distributed and Networked OS

Distributed Systems Concepts

A **distributed operating system** manages resources across multiple networked computers, making them appear as a single system. Key challenges: **consensus** (agreeing on values despite failures), **fault tolerance** (continuing despite node failures), **consistency** (all nodes see the same data), and **partition tolerance** (functioning despite network splits).

CAP Theorem

Eric Brewer's theorem: a distributed system can provide at most 2 of 3 guarantees: **Consistency** (all nodes see the same data), **Availability** (every request gets a response), **Partition tolerance** (system works despite network partitions). Since partitions are inevitable in real networks, the practical choice is between CP (consistent but may be unavailable) and AP (available but may be inconsistent).

Consensus Algorithms

Algorithm	Key Idea	Used By
Paxos	Multi-phase voting for single value agreement	Google Chubby, Spanner
Raft	Leader-based, understandable consensus	etcd, Consul, CockroachDB
ZAB	Zookeeper Atomic Broadcast	Apache ZooKeeper

Remote Procedure Call (RPC)

RPC lets a program call a function on a remote machine as if it were local. The client stub marshals arguments, sends them over the network, the server stub unmarshals and calls the function, then returns the result. Modern implementations: gRPC (Google, uses Protocol Buffers), Thrift (Facebook).

Distributed File Systems

Examples: NFS (Network File System — simple, stateless, widely used), GFS/HDFS (Google File System / Hadoop Distributed File System — optimized for large sequential reads), Ceph (POSIX-compliant, scalable, used in OpenStack).

Chapter 28

Real-Time Operating Systems (RTOS)

What Makes an OS Real-Time?

An RTOS guarantees that tasks complete within specified **deadlines**. It is not about speed — it is about predictability. A general-purpose OS like Linux can be fast on average but has unpredictable worst-case latencies (due to garbage collection, page faults, non-preemptible kernel sections). An RTOS bounds the worst case.

Hard vs Soft Real-Time

Type	Deadline	Miss Consequence	Examples
Hard	Absolute	System failure	Aircraft control, pacemaker, ABS brakes
Firm	Important	Useless result but no disaster	Radar tracking, robotics
Soft	Desirable	Degraded quality	Video streaming, gaming, VoIP

RTOS Design Principles

- **Priority-based preemptive scheduling**: The highest-priority ready task always runs immediately.
- **Bounded interrupt latency**: Time from interrupt to handler is bounded and known.
- **Priority inheritance**: If a low-priority task holds a resource needed by a high-priority task, temporarily boost the low-priority task to prevent **priority inversion**.
- **Minimal kernel**: Less code = less unpredictability.
- **No virtual memory / No paging**: Page faults are unpredictable. RTOS often uses static memory allocation.

Examples

FreeRTOS: Open source, <10 KB kernel, runs on microcontrollers (ESP32, STM32). Used in IoT devices. **QNX**: Commercial microkernel RTOS, used in automotive (BlackBerry QNX), medical devices. **VxWorks**: Used in NASA Mars rovers, Boeing 787. **Zephyr**: Linux Foundation project for IoT, supports many architectures. **PREEMPT_RT**: Linux kernel patch set that makes Linux a soft/firm real-time OS by making kernel code fully preemptible.

Chapter 29

Multiprocessor and Multicore OS Design

SMP Architecture

In **Symmetric Multiprocessing (SMP)**, all CPUs are equal and share main memory. The OS runs on all CPUs. Challenges: cache coherency (if CPU1 writes to address X, CPU2's cached copy is stale), lock contention (kernel locks become bottlenecks with many CPUs), and scheduling fairness.

Cache Coherency Protocols

MESI protocol (used by Intel): each cache line is in one of four states: **Modified** (dirty, only copy), **Exclusive** (clean, only copy), **Shared** (clean, multiple copies), **Invalid** (stale). When CPU1 writes, other CPUs' copies are invalidated. This is done via the cache coherency bus (snooping) or directory-based protocols (for NUMA).

Kernel Locking Strategies

Strategy	Description	Scalability
Big Kernel Lock (BKL)	One lock for entire kernel	Terrible (Linux removed BKL in 2011)
Fine-grained locking	Separate lock per data structure	Good, but complex and deadlock-prone
Read-Copy-Update (RCU)	Readers lock-free, writers defer freeing	Excellent for read-heavy workloads
Per-CPU data	Each CPU has its own copy	Perfect (no sharing = no contention)

Scalability Challenges

As core count increases (64, 128, 256+), traditional locking becomes a bottleneck. Solutions: per-CPU data structures (each CPU has its own run queue, memory allocator, etc.), RCU (read-copy-update for lock-free reads), and avoiding shared state altogether. The Linux kernel scales well to hundreds of cores through these techniques.

Chapter 30

OS Security In Depth

Defense in Depth

Modern OS security uses multiple layers: hardware (NX bit, SMEP, SMAP), kernel (ASLR, stack canaries, seccomp), and user space (sandboxing, capabilities, mandatory access control). No single layer is sufficient.

Address Space Layout Randomization (ASLR)

ASLR randomizes where code, stack, heap, and libraries are loaded in memory. An attacker cannot predict addresses, making exploitation harder. On Linux, enabled by default since kernel 2.6.12. Kernel ASLR (KASLR) randomizes kernel code location.

Mandatory Access Control (MAC)

Unlike DAC (discretionary — owner sets permissions), **MAC** enforces system-wide policies regardless of owner preferences. **SELinux** (Security-Enhanced Linux, developed by NSA): every process and file has a security context; the kernel checks a policy database for every access. **AppArmor**: path-based MAC, simpler than SELinux.

Secure Boot and Measured Boot

Secure Boot (UEFI): the firmware only loads bootloaders signed by trusted keys. Prevents bootkit malware. **Measured Boot**: each boot stage is measured (hashed) and stored in the TPM (Trusted Platform Module). Remote servers can verify (attest) that a machine booted a known-good configuration.

Sandboxing

Restrict what a process can do: **seccomp** (Linux): filter system calls — a sandboxed process can only call a whitelist of syscalls. Used by Chrome, Docker. **Pledge/Unveil** (OpenBSD): promise to only use certain syscall groups. **Landlock** (Linux 5.13+): unprivileged sandboxing of file access.

PART IV

PRO / EXPERT

Staff-level judgment and craft

Chapter 31

Linux Kernel Deep Dive

Kernel Source Structure

```
linux/
  arch/        # Architecture-specific code (x86, ARM, RISC-V)
  block/       # Block I/O layer
  drivers/     # All device drivers (~60% of kernel code)
  fs/          # File systems (ext4, btrfs, xfs, nfs, etc.)
  include/     # Header files
  init/        # Kernel initialization (main.c -> start_kernel())
  kernel/      # Core: scheduling, signals, locking, time
  mm/          # Memory management (page allocator, slab, mmap)
  net/          # Networking stack (TCP/IP, sockets)
  security/    # Security modules (SELinux, AppArmor)
  tools/       # User-space tools (perf, bpftool)
```

Process Representation: task_struct

Every process/thread in Linux is represented by a **task_struct** (~6 KB). It contains: PID, state, scheduling info (vruntime, policy, priority), memory descriptor (mm_struct with page tables), file descriptor table, signal handlers, cgroup membership, namespace pointers, and more. The kernel maintains a doubly-linked list of all tasks and a hash table for PID lookup.

Memory Allocators

Allocator	Purpose	Granularity
Buddy allocator	Physical page allocation	Pages (4 KB)
SLAB/SLUB/SLOB	Kernel object allocation	Bytes (cache-friendly)
vmalloc	Virtually contiguous allocation	Pages (non-contiguous phys)
kmalloc	Physically contiguous allocation	Bytes (backed by SLAB)
CMA	Contiguous memory for DMA	Large contiguous regions

The Scheduler: CFS Internals

CFS uses a red-black tree keyed by vruntime. The leftmost node (smallest vruntime) runs next — O(1) to pick (cached pointer). When a task runs for delta nanoseconds, its vruntime increases by delta * (NICE_0_LOAD / task_weight). High-priority tasks have higher weight, so their vruntime grows slower. The scheduler's time complexity: O(log n) for insertion, O(1) for picking the next task.

Chapter 32

Windows NT Kernel Architecture

Architecture Overview

Windows NT uses a **hybrid kernel** architecture. The kernel has two layers: (1) the **NT kernel** (ntoskrnl.exe) — thread scheduling, interrupt handling, synchronization, and (2) the **Executive** — higher-level services: object manager, I/O manager, memory manager, process manager, security reference monitor. The **Hardware Abstraction Layer (HAL)** sits below the kernel, abstracting platform-specific details.

Key Components

Component	Role
Object Manager	Manages all kernel objects (files, processes, mutexes) with handles
I/O Manager	Handles all I/O through a layered driver model (IRPs)
Memory Manager	Virtual memory, paging, section objects, copy-on-write
Process Manager	Process/thread creation, job objects
Security Ref Monitor	Access checks using tokens and ACLs
Registry	Hierarchical configuration database (kernel-managed)
Cache Manager	Unified file cache integrated with memory manager

Windows Scheduling

Windows uses a priority-based preemptive scheduler with 32 priority levels (0-31). Levels 16-31 are real-time; 0-15 are dynamic. The scheduler boosts priority temporarily for interactive threads (e.g., window receiving input). Windows also supports User-Mode Scheduling (UMS) and thread pools for efficient concurrency.

I/O Request Packets (IRPs)

Windows I/O uses a layered model. When an application calls ReadFile(), the I/O Manager creates an **IRP** (I/O Request Packet) and passes it down through a stack of drivers. Each driver can process, modify, or pass the IRP along. When the bottom driver completes, the IRP travels back up. This enables filter drivers (antivirus, encryption) to intercept I/O transparently.

Chapter 33

GPU Computing and CUDA OS Integration

GPU Architecture for OS Engineers

A GPU has thousands of simple cores organized into **Streaming Multiprocessors (SMs)**. Each SM has 32-128 cores, shared memory, and a warp scheduler. A **warp** is a group of 32 threads executing the same instruction in lockstep (SIMT). GPUs are throughput-optimized: they hide memory latency by switching between warps rather than caching like CPUs.

OS Support for GPUs

The OS must: (1) manage GPU memory (allocate/free device memory, map DMA buffers), (2) schedule GPU tasks (submit work to command queues, handle preemption), (3) manage GPU contexts (each process gets its own GPU context with isolated address space). On Linux, the DRM (Direct Rendering Manager) subsystem manages GPU resources. NVIDIA uses a proprietary kernel module.

CUDA Programming Model

```
// CUDA kernel: vector addition
__global__ void add(float *a, float *b, float *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}
// Host code:
int main() {
    float *d_a, *d_b, *d_c;
    int n = 1000000;
    cudaMalloc(&d_a, n * sizeof(float));
    cudaMalloc(&d_b, n * sizeof(float));
    cudaMalloc(&d_c, n * sizeof(float));
    // Copy data to GPU...
    add<<<(n+255)/256, 256>>>(d_a, d_b, d_c, n);
    cudaDeviceSynchronize();
    // Copy result back...
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
}
```

Unified Memory

CUDA Unified Memory (cudaMallocManaged) creates memory accessible from both CPU and GPU. The driver automatically migrates pages between CPU and GPU memory on demand. Simplifies programming but may hurt performance due to migration overhead. NVIDIA's newer GPUs support hardware page faulting for automatic migration.

Chapter 34

AI/ML Workloads and OS Scheduling

AI Workload Characteristics

AI workloads are fundamentally different from traditional workloads: (1) **Compute-bound**: matrix multiplications dominate (GEMM), favoring GPUs/TPUs. (2) **Memory-bandwidth bound**: large model parameters must be streamed through compute units. (3) **Distributed**: models are split across multiple GPUs/nodes. (4) **Long-running**: training runs last hours to weeks.

OS Challenges for AI

Challenge	Description	OS Solution
GPU scheduling	Multiple users sharing GPUs	MPS, MIG (Multi-Instance GPU), time-slicing
Memory management	Models larger than GPU memory	Unified memory, gradient checkpointing
Data pipeline	Feed data fast enough to GPU	Huge page TLB, io_uring, direct I/O
Distributed training	Synchronize gradients across nodes	RDMA (bypass kernel network stack)
Fault tolerance	Recover from node failures	Checkpoint/restart, elastic training

NVIDIA MIG (Multi-Instance GPU)

Multi-Instance GPU partitions a single GPU (A100, H100) into up to 7 isolated instances, each with its own memory, cache, and compute resources. The OS sees each instance as a separate GPU. This enables secure multi-tenant GPU sharing in cloud environments.

On-Device AI

Modern phones and edge devices have Neural Processing Units (NPUs). The OS must schedule inference tasks on NPU, GPU, or CPU based on model requirements, power constraints, and latency targets. Android's NNAPI and Apple's Core ML handle this abstraction.

Chapter 35

Storage Stack: RAID, NVMe, io_uring

RAID Levels

Level	Technique	Min Disks	Capacity	Fault Tolerance
RAID 0	Striping (no redundancy)	2	N × disk	None
RAID 1	Mirroring	2	N/2 × disk	1 disk failure
RAID 5	Striping + distributed parity	3	(N-1) × disk	1 disk failure
RAID 6	Striping + double parity	4	(N-2) × disk	2 disk failures
RAID 10	Mirror + stripe	4	N/2 × disk	1 per mirror pair

NVMe

NVMe (Non-Volatile Memory Express) is the protocol for modern SSDs connected via PCIe. Unlike AHCI (designed for spinning disks with one command queue), NVMe supports **64K queues with 64K commands each**. This enables massive parallelism. NVMe SSDs can achieve 7+ GB/s sequential reads and 1M+ IOPS random reads. The OS NVMe driver submits commands directly to hardware queues with minimal kernel involvement.

io_uring

Linux's modern async I/O interface (since kernel 5.1, 2019). Previous interfaces (POSIX AIO, libaio) were limited. io_uring uses two ring buffers shared between user space and kernel: a **submission queue (SQ)** and a **completion queue (CQ)**. User space writes I/O requests to SQ, kernel processes them and posts completions to CQ. Benefits: (1) zero-copy submission, (2) batch submission, (3) no system call needed per I/O (poll mode), (4) supports files, sockets, timers, and more.

```
// io_uring simplified flow:
// 1. Setup: io_uring_setup() -> shared memory rings
// 2. Submit: write SQE (submission queue entry) to SQ ring
// 3. Notify: io_uring_enter() or poll mode (no syscall)
// 4. Complete: read CQE (completion queue entry) from CQ ring
// Result: up to 10x fewer syscalls than traditional I/O
```

Chapter 36

Network Stack Internals

Linux Network Stack

```
// Packet flow (incoming):
NIC -> DMA to ring buffer -> Hard IRQ -> NAPI softirq
  -> Driver -> netfilter/iptables -> IP layer
  -> TCP/UDP -> Socket buffer -> Application read()
```

Key Components

• **sk_buff**: The kernel's packet buffer structure. Contains pointers to protocol headers, data, and metadata. Designed for zero-copy header manipulation.

• **NAPI (New API)**: Combines interrupts and polling. On light load, use interrupts. On heavy load, switch to polling to avoid interrupt storms (livelock).

• **Netfilter**: The kernel's packet filtering framework. Hooks at 5 points: PRE_ROUTING, INPUT, FORWARD, OUTPUT, POST_ROUTING. iptables/nftables are user-space tools that configure Netfilter rules.

• **TCP congestion control**: Pluggable algorithms: Reno, CUBIC (default Linux), BBR (Google, bandwidth-based). Each adapts sending rate differently to network conditions.

High-Performance Networking

Technique	Description	Use Case
XDP (eXpress Data Path)	eBPF programs on NIC driver, before kernel stack	DDoS mitigation, load balancing
DPDK	Bypass kernel entirely, poll NIC from user space	Telecom, NFV, trading
RDMA	Direct memory access over network, no CPU	HPC, distributed training
TCP_FASTOPEN	Send data in SYN packet	Reduce web latency
Zero-copy (sendfile)	Transfer file to socket without user-space copy	Web servers, file transfers

Chapter 37

Performance Analysis and Profiling

The USE Method

Brendan Gregg's **USE Method**: for every resource (CPU, memory, disk, network), check three metrics: **Utilization** (% time busy), **Saturation** (degree of queuing — work that cannot be serviced), **Errors** (error count). This systematically identifies bottlenecks.

Linux Performance Tools

Tool	What It Shows	Layer
top / htop	CPU, memory per process	Process
vmstat	Virtual memory, CPU, I/O summary	System-wide
iostat	Disk I/O throughput, latency, utilization	Block I/O
perf	CPU profiling, cache misses, branch mispredictions	CPU/Hardware
perf record + flamegraph	Call stack sampling → flame graph visualization	CPU
strace	Trace system calls	Syscall
bpftrace / eBPF	Dynamic tracing of kernel and user functions	Any layer
ftrace	Kernel function tracing	Kernel
sar	Historical system activity reporting	System-wide

eBPF for Observability

eBPF (extended Berkeley Packet Filter) is a revolutionary kernel technology. It lets you run sandboxed programs inside the kernel without modifying kernel code or loading modules. Use cases: tracing, networking, security. Tools like bpftrace, BCC, and cilium are built on eBPF. Example: trace every process that calls open() with the filename and latency.

```
# bpftrace: trace file opens with latency
bpftrace -e '
  tracepoint:syscalls:sys_enter_openat {
    @start[tid] = nsecs;
    @fname[tid] = str(args->filename);
  }
  tracepoint:syscalls:sys_exit_openat /@start[tid]/ {
    printf("%s opened %s in %d us\n",
      comm, @fname[tid], (nsecs - @start[tid])/1000);
    delete(@start[tid]);
    delete(@fname[tid]);
  }
'
```

Chapter 38

Building a Minimal OS Kernel

What You Need

Building an OS kernel from scratch teaches more about OS concepts than any textbook. A minimal kernel needs: a bootloader, kernel entry point, console output, interrupt handling, and (optionally) a simple memory allocator and process scheduler.

Boot Sequence

```
// x86 boot sequence:
// 1. BIOS/UEFI loads bootloader from disk
// 2. Bootloader (GRUB) switches to protected mode (32-bit)
// 3. Bootloader loads kernel image into memory
// 4. Bootloader jumps to kernel entry point
// 5. Kernel sets up GDT, IDT, page tables
// 6. Kernel enables interrupts
// 7. Kernel initializes devices, scheduler
// 8. Kernel starts first user process (init)
```

Minimal Kernel Checklist

Component	What to Implement	Complexity
Console output	Write to VGA text buffer (0xB8000) or serial port	Easy
GDT/IDT	Global Descriptor Table, Interrupt Descriptor Table	Medium
Interrupt handling	Timer interrupt (PIT), keyboard handler	Medium
Physical memory	Bitmap allocator for page frames	Medium
Virtual memory	Page tables, map kernel into high memory	Hard
Process/threads	task_struct, context switch in assembly	Hard
Scheduler	Round-robin with a ready queue	Medium
System calls	Software interrupt handler, dispatch table	Medium
File system	Read-only ramdisk (initrd)	Medium
User space	Switch to ring 3, run a simple program	Hard

Recommended Projects

- **OSDev Wiki** (wiki.osdev.org): The definitive resource for OS development.
- **xv6** (MIT): A simple Unix-like OS for teaching. ~10K lines of C. Runs on x86/RISC-V.
- **Writing an OS in Rust** (os.phil-opp.com): Modern approach using Rust's safety guarantees.
- **MINIX 3**: Andrew Tanenbaum's microkernel OS, designed for studying OS design.

Chapter 39

Emerging Research: eBPF, Unikernels, WASM

eBPF: Programmable Kernel

eBPF has transformed Linux from a fixed kernel into a **programmable platform**. eBPF programs are verified (the kernel checks they terminate, don't access invalid memory, etc.) and JIT-compiled to native code. Use cases span networking (Cilium replaces iptables), security (Falco detects anomalous syscalls), and observability (Pixie monitors Kubernetes). eBPF is becoming the standard way to extend the kernel safely.

Unikernels

A **unikernel** compiles the application, libraries, and kernel together into a single-purpose image that runs directly on a hypervisor or bare metal. No shell, no users, no unnecessary code. Benefits: tiny attack surface (MirageOS HTTP server: ~5 MB), fast boot (milliseconds), minimal memory. Drawback: no debugging tools (no SSH, no top). Examples: MirageOS (OCaml), Unikraft (C/C++, Linux-compatible), OSv (runs unmodified Linux binaries).

WebAssembly (WASM) in the Kernel

WASM is a portable bytecode originally designed for web browsers. Its sandboxed execution model is being explored as a kernel extension mechanism — an alternative to eBPF. WASM modules can be loaded into the kernel to implement custom scheduling policies, network protocols, or file system handlers. Still experimental but promising for cross-platform kernel extensibility.

Rust in the Kernel

Since Linux 6.1 (December 2022), Rust is an officially supported language for kernel development. Rust's ownership system prevents data races and use-after-free bugs at compile time — the most common classes of kernel vulnerabilities. The first Rust drivers are appearing in production (Android's Binder IPC). This is the most significant change to kernel development methodology in decades.

Chapter 40

OS Design Patterns and Trade-offs

Fundamental Trade-offs

Trade-off	Option A	Option B	Judgment Call
Monolithic vs Micro	Fast (no IPC)	Secure (fault isolation)	Linux chose monolithic + modules
Preemptive vs Cooperative	Fair, responsive	Simple, efficient	All modern OSes: preemptive
Eager vs Lazy allocation	Simple, predictable	Memory efficient	Most OSes: lazy (demand paging)
Spin vs Sleep	Low latency, wastes CPU	Saves CPU, higher latency	Spin for < 1µs, sleep otherwise
Consistency vs Performance	Correct but slow	Fast but complex	Depends on workload
Complexity vs Security	More features	Smaller attack surface	Principle of least privilege

Design Patterns

- **Layering**: Stack abstractions (hardware → driver → file system → VFS → application). Each layer only talks to adjacent layers.
- **Indirection**: Virtual memory, virtual file systems, virtual devices. 'Any problem in CS can be solved by adding another level of indirection.'
- **Caching**: TLB, page cache, dentry cache, inode cache. The OS is fundamentally a cache management system.
- **Batching**: Deferred work, delayed allocation, write-back caching. Do work in bulk to amortize overhead.
- **Concurrency control**: Fine-grained locking, RCU, lock-free structures. The most difficult part of kernel design.

The Future of Operating Systems

Key trends: (1) **Hardware heterogeneity**: CPUs + GPUs + NPUs + FPGAs — the OS must orchestrate diverse accelerators. (2) **Security as a first-class concern**: Confidential computing (AMD SEV, Intel TDX) enables encrypted VMs that even the hypervisor cannot read. (3) **Kernel programmability**: eBPF and WASM allow safe kernel extension without recompilation. (4) **Rust and formal verification**: Memory-safe languages and mathematical proofs for critical kernel components. (5) **AI-native OS**: Operating systems that natively manage AI workloads, model serving, and inference scheduling.

Closing Thought

Operating systems are the foundation of all computing. They are among the most complex software systems ever built — Linux alone has over 30 million lines of code, contributed by thousands of developers over 35 years. Mastering OS concepts gives you a deep understanding of how software really works, from the electrons in transistors to the applications on your screen. This knowledge is timeless: the fundamental problems of resource management, concurrency, and abstraction will remain relevant regardless of how hardware evolves.

QUICK REFERENCE

Decision guides, cheat sheets, and at-a-glance comparisons

Quick Reference

OS Concepts Cheat Sheet

Process States

```
New -> Ready -> Running -> Terminated
            \-> Waiting -> Ready
Running -> Ready (preempted by scheduler)
Running -> Waiting (I/O request)
```

Scheduling Algorithms

Algorithm	Type	Best For
FCFS	Non-preemptive	Simple batch jobs
SJF	Non-preemptive	Minimizing avg wait time (needs prediction)
SRTF	Preemptive SJF	Minimizing avg wait time (preemptive)
Round Robin	Preemptive	Interactive systems (fair time sharing)
Priority	Either	Mixed workloads (with aging)
MLFQ	Preemptive	General purpose (adapts to behavior)
CFS	Preemptive	Linux default (fair, O(log n))
EDF	Preemptive	Real-time (optimal on uniprocessor)

Memory Hierarchy

```
Registers  (~0.3ns)  ->  L1 Cache (~1ns)  ->  L2 Cache (~4ns)
  ->  L3 Cache (~10ns)  ->  RAM (~100ns)  ->  SSD (~25us)
  ->  HDD (~5ms)
Speed gap: L1 to RAM = ~100x, RAM to SSD = ~250x, RAM to HDD = ~50000x
```

Synchronization Quick Reference

Primitive	Use When	Behavior When Contended
Mutex	Protecting shared data	Block (sleep)
Spinlock	Very short critical sections in kernel	Spin (busy-wait)
Semaphore(N)	Limiting concurrent access to N	Block (sleep) when count < 0
Read-Write Lock	Many readers, few writers	Readers share, writer exclusive
Condition Variable	Waiting for a condition	Sleep until signaled
Barrier	Synchronize N threads at a point	Block until all N arrive

Deadlock Conditions (All 4 Required)

```
1. Mutual Exclusion     - Resource held exclusively
2. Hold and Wait        - Hold one, wait for another
3. No Preemption        - Cannot force release
4. Circular Wait        - A waits for B waits for A
```

```
Break ANY ONE to prevent deadlock.
```

Page Replacement

Algorithm	Evicts	Practical?
OPT	Page used farthest in future	No (theoretical best)
FIFO	Oldest page	Yes (but has Belady's anomaly)
LRU	Least recently used	Expensive (hardware needed)
Clock	First page with reference bit = 0	Yes (approximates LRU)

Key Formulas

```
Effective Access Time (EAT) with TLB:
  EAT = hit_rate * (TLB_time + memory_time) +
        (1 - hit_rate) * (TLB_time + N * memory_time)
  where N = number of page table levels
Effective Access Time with paging:
  EAT = (1 - p) * memory_time + p * page_fault_time
  where p = page fault rate
Disk Bandwidth:
  Bandwidth = Transfer_size / (Seek_time + Rotational_latency + Transfer_time)
```

Linux Commands Quick Reference

```
ps aux          # List all processes
top / htop      # Interactive process monitor
kill -9 PID     # Force kill process
strace cmd      # Trace system calls
lsof            # List open files
vmstat 1        # Virtual memory stats every 1s
iostat -x 1     # Disk I/O stats every 1s
free -h         # Memory usage
df -h           # Disk space
perf top        # CPU profiling
dmesg           # Kernel log messages
```